

Django Vue Lab

Лекция 10. Архитектура веб-приложений

Черепанов И.Н. Филиппьев В.А.

Содержание

Архитектура	3
OSE/RM	10
Системная и прикладная разработка	13
Декомпозиция	17
Модули Django	28
Модули Vue	38
API как граница	44
Слои и модули	49
Практика	51

Архитектура

Что такое архитектура

Архитектура приложения – набор решений, которые определяют:

- из каких частей состоит система
- как части связаны между собой
- где хранятся данные
- где выполняется бизнес-логика
- как приложение изменяется без полного переписывания

Архитектура нужна не для красоты, а чтобы проект можно было понимать, менять и защищать.

Главная сложность

На момент проектирования у нас почти никогда нет полной информации.

- требования меняются
- пользователь сам не всегда знает, что ему нужно
- часть проблем видна только после прототипа
- реальные данные отличаются от учебных примеров
- интеграции и ограничения появляются позже
- команда узнает предметную область в процессе работы

Поэтому архитектура должна не угадывать будущее, а оставлять место для изменений.

Хорошая архитектура

- Понятно, где искать код
- Изменения локальны
- Ошибки не ломают всю систему
- Части можно тестировать отдельно
- Новую функциональность можно добавить без хаоса
- Команда может работать параллельно

Плохая архитектура

- `views.py` на 2000 строк
- одна модель знает обо всем
- один компонент Vue управляет всей страницей
- бизнес-правила спрятаны в шаблонах
- API возвращает случайную структуру данных
- любая правка ломает соседнюю функцию

Веб-приложение как система

```
graph TD; A[Пользователь] --> B[Браузер / Vue]; B --> C[HTTP / REST API]; C --> D[Django / DRF]; D --> E[ORM]; E --> F[База данных];
```

Пользователь
|
Браузер / Vue
|
HTTP / REST API
|
Django / DRF
|
ORM
|
База данных

Каждый слой решает свою задачу. Проблемы начинаются, когда слой берет на себя чужую ответственность.

Django + Vue

- Django хранит данные и правила предметной области
- DRF публикует API
- Vue отвечает за пользовательский интерфейс
- браузер хранит временное состояние интерфейса
- база данных хранит долговременное состояние системы

Важно: frontend не должен быть единственным местом проверки правил.

OSE/RM

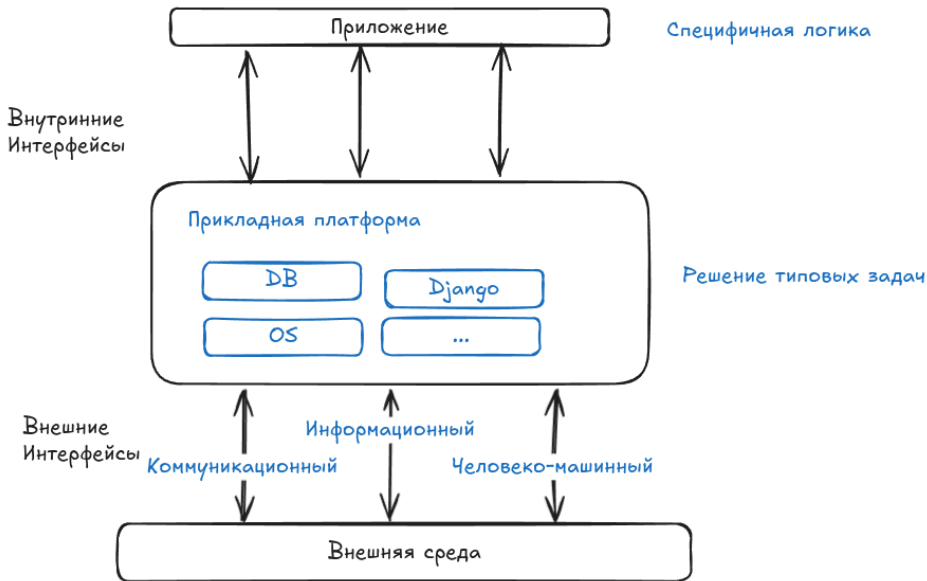
Открытые системы

Открытая система – система, построенная на открытых стандартах и спецификациях.

- HTTP
- HTML, CSS, JavaScript
- JSON
- SQL
- OpenAPI
- Docker image format

Открытые стандарты позволяют заменять части системы без полной переделки.

OSE/RM Open Systems Environment



Системная и прикладная разработка

Системная и прикладная разработка

Системная разработка создает основу, на которой работают другие программы.

- операционные системы
- драйверы
- компиляторы и интерпретаторы
- базы данных и брокеры сообщений
- сетевые протоколы и серверы

Здесь главные вопросы: ресурсы, производительность, надежность, совместимость и взаимодействие с железом или платформой.

Прикладная разработка

Прикладная разработка решает задачи конкретной предметной области.

- интернет-магазин: каталог, корзина, заказы, оплата
- университет: студенты, группы, расписание, оценки
- клиника: пациенты, приемы, назначения, счета

В этой лекции мы рассматриваем архитектуру именно на уровне предметной области: как выделять сущности, модули, бизнес-правила и границы ответственности в прикладном веб-приложении.

Термины из предметной области

В прикладной разработке не нужно начинать со сложных абстракций.

Хороший код говорит на языке предметной области:

- Order, а не BusinessTransactionEntity
- Cart, а не TemporaryProductAccumulator
- Student, Group, Lesson, а не универсальные Object, Record, Item

Названия классов, функций, модулей и API должны быть максимально близки к названиям предметов, действий и правил, с которыми работает пользователь.

Декомпозиция

Декомпозиция

Декомпозиция – разбиение большой задачи на небольшие части.

Цель: сделать систему понятной человеку.

Не нужно делать много файлов ради файлов. Нужно выделять части, которые имеют собственную ответственность.

Разделяйте по предметной области

Хорошие модули часто соответствуют словам из предметной области.

- пользователи
- каталог
- корзина
- заказы
- оплата
- отзывы
- уведомления

Если вы не можете назвать модуль простым существительным, граница выбрана плохо.

Разделяйте по скорости изменений

Части, которые часто меняются по разным причинам, лучше разделять.

- интерфейс меняется из-за UX
- API меняется из-за клиентов
- модели меняются из-за данных
- бизнес-правила меняются из-за требований
- инфраструктура меняется из-за развертывания

Смешивание этих причин создает хрупкий код.

Разделяйте интерфейс и реализацию

Интерфейс – как с модулем взаимодействуют.

Реализация – как он работает внутри.

POST /api/orders/	интерфейс
OrderSerializer	формат данных
create_order()	бизнес-операция
Order, OrderItem	хранение данных

Клиенту API не важно, сколько таблиц используется внутри.

Не дробите слишком рано

Плохая декомпозиция тоже вредна.

- слишком много маленьких файлов
- непонятные абстракции
- классы без реальной логики
- сложно проследить выполнение запроса
- невозможно быстро изменить поведение

Начинайте просто. Выделяйте модуль, когда появилась реальная ответственность.

Сцепление и связность

Связность – насколько код внутри модуля относится к одной задаче.

Сцепление – насколько сильно модуль зависит от других модулей.

Хорошо: высокая связность, низкое сцепление.

```
catalog зависит от users -- нормально  
catalog зависит от cart internals -- тревожный признак
```

Инверсия зависимостей

Инверсия зависимостей – принцип, при котором важная бизнес-логика не зависит напрямую от технических деталей.

Плохо:

логика заказа напрямую знает, как отправлять email, писать в файл, ходить во внешний платежный сервис.

Лучше:

логика заказа вызывает понятный интерфейс:

`notify_user()`, `charge_payment()`, `save_report()`.

Детали должны зависеть от правил приложения, а не правила приложения от деталей.

Зачем это нужно

- проще менять реализацию
- проще тестировать бизнес-логику
- меньше зависимость от внешних сервисов
- меньше кода ломается при изменении инфраструктуры
- проще поддерживать разные варианты поведения

Например, в тестах можно заменить реальную оплату на фейковую функцию.

Пример в Django

```
def create_order(user, cart, send_notification):
    order = Order.objects.create(user=user, status="new")

    for item in cart.items.select_related("product"):
        OrderItem.objects.create(
            order=order,
            product=item.product,
            price=item.product.price,
            quantity=item.quantity,
        )

    send_notification(user, order)
    return order
```

`create_order()` не знает, будет уведомление через email, Telegram или тестовую заглушку.

Пример во Vue

```
export function createProductPage(api) {  
  return {  
    async loadProducts(params) {  
      return await api.fetchProducts(params)  
    }  
  }  
}
```

Компоненту не важно, откуда пришли данные: с настоящего API, mock-сервера или тестовой заглушки.

Модули Django

Django app

Django app – модуль приложения с собственной областью ответственности.

```
project/  
  config/  
  users/  
  catalog/  
  orders/  
  reviews/
```

App не обязан быть маленьким. Он должен быть понятным.

Что класть в app

catalog/	
models.py	Product, Category
serializers.py	ProductSerializer
services.py	Domain logig
views.py	ProductViewSet
urls.py	/api/products/
admin.py	настройка админки
tests.py	тесты каталога

Если файл становится слишком большим, его можно заменить пакетом.

Когда делать новый app

- есть отдельная предметная область
- есть собственные модели
- есть собственные API endpoints
- есть отдельные права доступа
- команда может работать над частью независимо

Не стоит делать app для каждой таблицы.

Пример разбиения магазина

users	регистрация, профиль, роли
catalog	товары, категории, остатки
cart	корзина пользователя
orders	оформление и история заказов
payments	платежи и статусы оплаты
reviews	отзывы и рейтинги

Такое разбиение понятно бизнесу и разработчикам.

Модель данных

Модель – не просто таблица. Это часть предметной области.

```
class Order(models.Model):
    user = models.ForeignKey(User, on_delete=models.PROTECT)
    status = models.CharField(max_length=32)
    created_at = models.DateTimeField(auto_now_add=True)

class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE)
    product = models.ForeignKey(Product, on_delete=models.PROTECT)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    quantity = models.PositiveIntegerField()
```

Цена сохраняется в заказе, потому что цена товара может измениться позже.

Где держать бизнес-логику

В маленьком проекте можно начать с `views.py` и `models.py`.

Когда логики становится больше, выделяйте сервисные функции.

```
def create_order(user, cart):
    if cart.is_empty():
        raise ValueError("Cart is empty")

    order = Order.objects.create(user=user, status="new")
    for item in cart.items.select_related("product"):
        OrderItem.objects.create(
            order=order,
            product=item.product,
            price=item.product.price,
            quantity=item.quantity,
        )
    return order
```

View не должен делать все

```
class OrderViewSet(ModelViewSet):  
    serializer_class = OrderSerializer  
  
    def perform_create(self, serializer):  
        serializer.instance = create_order(  
            user=self.request.user,  
            cart=self.request.user.cart,  
        )
```

View связывает HTTP-запрос с операцией приложения.

Сериализаторы

Сериализатор – граница между внешними данными и внутренней моделью.

- описывает формат API
- проверяет входные данные
- скрывает внутренние поля
- управляет вложенными объектами

Не превращайте сериализатор в место всей бизнес-логики.

Права доступа

Права – часть архитектуры.

Кто может видеть объект?
Кто может создать объект?
Кто может изменить объект?
Кто может удалить объект?

Проверки должны быть на backend. Проверка во Vue нужна только для удобства интерфейса.

Модули Vue

Vue как приложение

Во Vue тоже нужна декомпозиция.

```
src/  
  views/  
  components/  
  api/  
  stores/  
  router/
```

Page знает сценарий страницы. Component отвечает за переиспользуемый элемент. API module знает HTTP.

View и component

```
views/ProductListPage.vue  
components/ProductCard.vue  
components/ProductFilters.vue  
api/products.js
```

View - Страница для пользователя, подключается в router.

Component - размещаются на views.

API слой во Vue

```
export async function fetchProducts(params) {  
  const response = await fetch(`/api/products/?${params}`)  
  
  if (!response.ok) {  
    throw new Error("Cannot load products")  
  }  
  
  return await response.json()  
}
```

Компоненты не должны везде вручную собирать URL и разбирать ошибки.

Состояние frontend

Не все данные нужно класть в global store.

- локальное состояние компонента
- состояние страницы
- состояние маршрута
- глобальное состояние пользователя
- кэш данных API

Чем глобальнее состояние, тем труднее понимать приложение.

Маршруты

Маршруты – часть архитектуры интерфейса.

<code>/products</code>	список товаров
<code>/products/:id</code>	карточка товара
<code>/cart</code>	корзина
<code>/orders</code>	мои заказы
<code>/orders/:id</code>	детали заказа

URL должен быть понятной точкой входа в сценарий пользователя.

API как граница

API contract

API contract – договор между backend и frontend.

```
{  
  "id": 42,  
  "title": "Django REST Framework",  
  "price": "1500.00",  
  "is_available": true  
}
```

Если договор меняется, меняются обе стороны.

Ресурсы и действия

REST хорошо работает для ресурсов.

```
GET    /api/products/  
GET    /api/products/42/  
POST   /api/cart/items/  
PATCH /api/cart/items/7/  
DELETE /api/cart/items/7/  
POST   /api/orders/
```

Не называйте URL глаголами без необходимости: `/api/do-create-good-order/`.

Ошибки API

Ошибки тоже часть контракта.

```
{  
  "detail": "Недостаточно товара на складе",  
  "code": "not_enough_stock"  
}
```

Frontend должен понимать, что показать пользователю.

Версионирование

Для учебного проекта достаточно аккуратно не ломать API без причины.

В реальных системах используют версии.

```
/api/v1/products/  
/api/v2/products/
```

Версия нужна, когда старые клиенты еще работают, а новый формат уже нужен.

Слои и модули

Одинаковая модульная структура

django

orders/

catalog/

models.py

serializers.py

services.py

views.py

urls.py

admin.py

tests.py

vue

src/

orders/

catalog/

components/

views/

router.js

api.js

stores.js

Практика

Как проектировать модуль

1. Опишите сценарии пользователя
2. Выпишите сущности предметной области
3. Определите связи между сущностями
4. Решите, какие операции нужны
5. Спроектируйте API
6. Разделите Django apps и Vue pages
7. Проверьте права доступа и ошибки

Архитектурные вопросы

- Где источник истины?
- Кто владеет данными?
- Что произойдет при ошибке?
- Можно ли заменить интерфейс?
- Можно ли протестировать правило отдельно?
- Что будет, если данных станет в 100 раз больше?
- Что будет, если появится второй тип пользователя?

Частые ошибки в проектах

- app называется main и содержит все
- модели не отражают предметную область
- права проверяются только на frontend
- API возвращает HTML вместо данных
- компоненты Vue напрямую знают структуру БД
- нет обработки пустых, ошибочных и загрузочных состояний
- нет единого подхода к URL

Итог

- Архитектура – это границы и ответственность
- OSE/RM помогает смотреть на систему слоями
- Декомпозиция должна идти от предметной области
- Django apps – модули backend
- Vue pages/components – модули frontend
- API – граница между frontend и backend
- Хороший модуль легко понять, изменить и проверить